Practical No.1

Aim:- Create different blinking patterns with LEDs by connecting to GPIO pins of Raspberry Pi.

Hardware Required:-

1. Raspberry Pi 3B+
2. Ethernet Cable (optional)
3. Monitor
4. HDMI to VGA convertor
5. Micro SD card (any class best is class 10)
6. Adaptor with 5v 2A
7. USB mouse
8. USB keyboard
9. 3 LED and 3 resistors
10. Breadboard
11. 4 Wires (Male-Female)

Software Required:-

1. Raspbian OS

Code:-

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)

leds = [17, 18, 27, 22, 23, 24, 25, 5]

for led in leds:
    GPIO.setup(led, GPIO.OUT)
    GPIO.output(led, GPIO.LOW)

try:
    # Pattern 1: All LEDs ON and OFF
    for i in range(5):
        for led in leds:
            GPIO.output(led, GPIO.HIGH)
        time.sleep(0.5)

        for led in leds:
            GPIO.output(led, GPIO.LOW)
        time.sleep(0.5)
```

```
    # Pattern 2: Left to Right
    for i in range(3):
        for led in leds:
            GPIO.output(led, GPIO.HIGH)
            time.sleep(0.2)
            GPIO.output(led, GPIO.LOW)

    # Pattern 3: Right to Left
    for i in range(3):
        for led in reversed(leds):
            GPIO.output(led, GPIO.HIGH)
            time.sleep(0.2)
            GPIO.output(led, GPIO.LOW)

    # Pattern 4: Alternate LEDs
    for i in range(6):
        for j, led in enumerate(leds):
            if j % 2 == 0:
                GPIO.output(led, GPIO.HIGH)
            else:
                GPIO.output(led, GPIO.LOW)
        time.sleep(0.4)

        for j, led in enumerate(leds):
            if j % 2 == 1:
                GPIO.output(led, GPIO.HIGH)
            else:
                GPIO.output(led, GPIO.LOW)
        time.sleep(0.4)

    # Pattern 5: LED Filling
    for i in range(len(leds)):
        GPIO.output(leds[i], GPIO.HIGH)
        time.sleep(0.2)

    time.sleep(1)

    for i in range(len(leds)):
        GPIO.output(leds[i], GPIO.LOW)
        time.sleep(0.2)

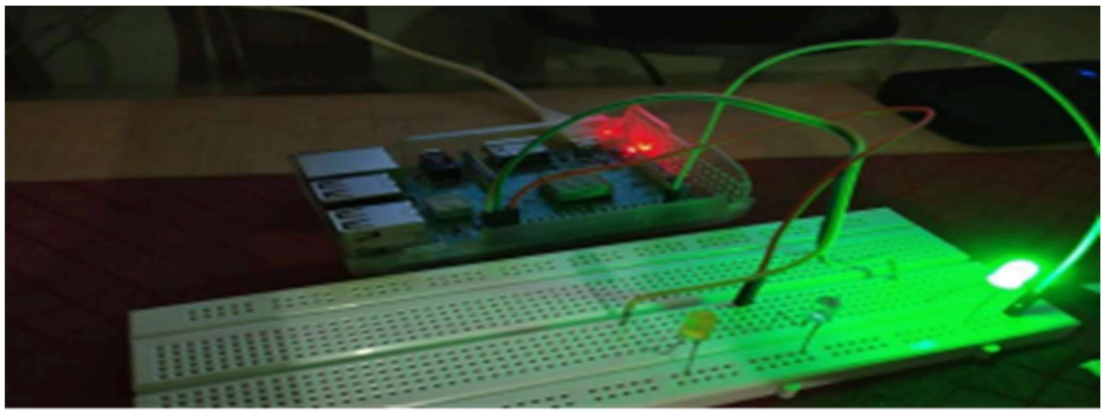
    # Pattern 6: Knight Rider
    for i in range(5):
        for led in leds:
            GPIO.output(led, GPIO.HIGH)
            time.sleep(0.08)
            GPIO.output(led, GPIO.LOW)

for led in reversed(leds):
```

```
GPIO.output(led, GPIO.HIGH)
time.sleep(0.08)
GPIO.output(led, GPIO.LOW)

finally:
GPIO.cleanup()
```

Output:-



Practical No.2

Aim:- Use Telegram to control Raspberry Pi GPIO.

Hardware Required:-

1. Raspberry Pi 3B+
2. Ethernet Cable
3. Monitor
4. HDMI to VGA convertor
5. Micro SD card (any class best is class 10)
6. Adaptor with 5v 2A
7. USB mouse
8. USB keyboard
9. Female – male jumper wires.
10. Bread Board
11. LED

Software Required:-

1. Raspbian OS
2. Thony python IDE
3. Telegram App

Procedure:-

1. Hardware Setup:
● Connect according to the figure.
PIN 37 (GPIO 26)□□ LED Anode
PIN 39 (GND) □□ LED cathode
*Black wire is the GND (ground)

2. Software Setup:
● Install Telegram in android phone.
● Search for contact “BotFather”
● Send him “/start” command
● Send him “/newbot”
● Give a name to your bot
● Give a username to the bot (It must be a unique one)
● On success you will be provided with a HTTP API token. (jot it down)
● In Raspberry Pi Terminal type the following…
➢ sudo apt update
➢ sudo apt upgrade
➢ sudo pip install telepot
or
➢ sudo pip install --break-system-packages telepot
● open Geany Programmer’s Editor and create a python script as given

below

Code:-

```
import time
import telepot
from telepot.loop import MessageLoop
import RPi.GPIO as GPIO

# ---------------- GPIO Setup ----------------
LED_PIN = 37      # Physical Pin 37

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BOARD)
GPIO.setup(LED_PIN, GPIO.OUT)
GPIO.output(LED_PIN, GPIO.LOW)

# ---------------- Telegram Bot ----------------
BOT_TOKEN = "8995068536:AAGgou8jclT-LI7rFMsJvsWQe8Tro3UPMBs"

telegram_bot = telepot.Bot(BOT_TOKEN)


def handle(msg):
    chat_id = msg['chat']['id']
    command = msg['text'].strip().lower()

    print("Chat ID:", chat_id)
    print("Received:", command)

    if command == "on":
        GPIO.output(LED_PIN, GPIO.HIGH)
        telegram_bot.sendMessage(chat_id, " 💡 LED is ON")

    elif command == "off":
        GPIO.output(LED_PIN, GPIO.LOW)
        telegram_bot.sendMessage(chat_id, " 💡 LED is OFF")

    elif command == "status":
        if GPIO.input(LED_PIN):
            telegram_bot.sendMessage(chat_id, "LED is ON")
        else:
            telegram_bot.sendMessage(chat_id, "LED is OFF")

    elif command == "help":
        telegram_bot.sendMessage(
            chat_id,
            "Available commands:\n"
```

```
            "on - Turn LED ON\n"
            "off - Turn LED OFF\n"
            "status - Check LED status"
        )

    else:
        telegram_bot.sendMessage(chat_id, "Unknown command. Type 'help'.")

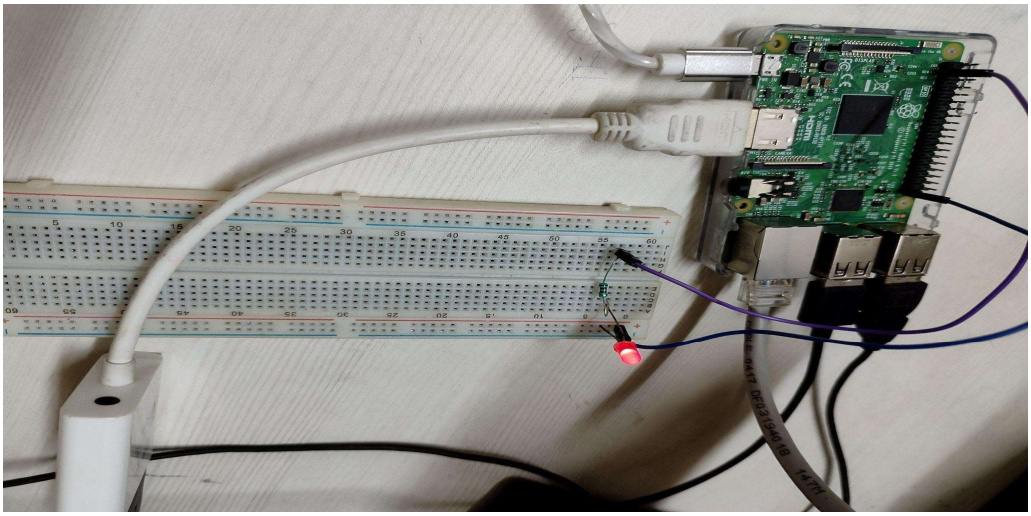

# Start bot
MessageLoop(telegram_bot, handle).run_as_thread()

print("Bot is running...")

try:
    while True:
        time.sleep(5)

except KeyboardInterrupt:
    print("Stopping...")
    GPIO.cleanup()
```

Output:-



Practical No.3

Aim:- Interface TM1637 with Raspberry Pi and Display current time.

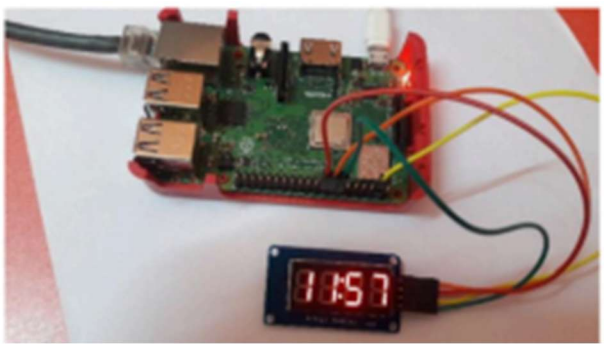
Hardware Required:-
1. Raspberry Pi 3B+
2. Ethernet Cable
3. Monitor
4. HDMI to VGI converter (required only in case of VGA cable)
5. adapter with 5V/2A
6. USB mouse
7. USB keyboard—-
8. 4 Digit 7 Segment Display

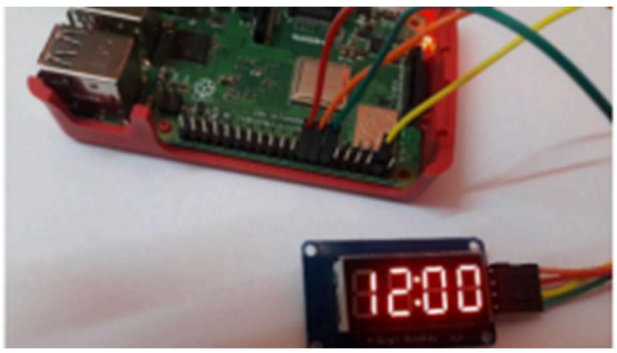
Procedure:-
1. Hardware Setup:
7 Segment Display Function Pin No(Raspberry Pi)
GND 6
VCC 1
CLK 11
DI0 13

Code:-

```
import tm1637
import time
# CLK = GPIO 27, DIO = GPIO 17
display = tm1637.TM1637(clk=27, dio=17)
display.brightness(7)
while True:
    now = time.localtime()
    # HH:MM
    hour = now.tm_hour
    minute = now.tm_min
    display.numbers(hour, minute, colon=True)
    time.sleep(1)
```

Output:-





Practical No.4

Aim:- Interface GPS module with Raspberry Pi.

NEO-6M → Raspberry Pi 3 B/B+
GPS NEO-6M Raspberry Pi GPIO Physical pin
VCC 3.3V 1
GND GND 6
TX GPIO 15 / RXD 10
RX GPIO 14 / TXD 8
Important: GPS TX → Pi RX, and GPS RX → Pi TX.

1. Enable UART

Run:-
sudo raspi-config

Go to:-
Interface Options → Serial Port

Set:-
Login shell over serial: No

Serial port hardware: Yes

Then reboot:-
sudo reboot

2. Install Python serial library
sudo apt update
sudo apt install -y python3-serial

3. Test the GPS
After connecting the module:

ls -l /dev/serial*
Usually you'll see something such as:

/dev/serial0

Then run:-
sudo cat /dev/serial0
You should eventually see NMEA messages similar to:

$GPGGA,...
$GPRMC,...
$GPGSV,...
If you see these messages, the GPS is communicating with the Raspberry Pi.

4. Python GPS program

```
import serial
import time

gps = serial.Serial(
    port="/dev/serial0",
    baudrate=9600,
    timeout=1
)

print("GPS started...")

try:
    while True:
        line = gps.readline().decode("ascii", errors="replace").strip()

        if line:
            print(line)

except KeyboardInterrupt:
    print("\nGPS stopped.")

finally:
    gps.close()
```

Run:

```
python3 gps.py
```

Note: The GPS may take several minutes to obtain a satellite fix, especially indoors. For the first test, place the GPS antenna where it has a clear view of the sky.

Practical No.5

Aim:- Interface ADS 1115 with Raspberry Pi to act as oscilloscope.

Hardware Required:-
1. Raspberry Pi 3B+
2. Ethernet Cable
3. Monitor
4. HDMI to VGI convertor
5. Adaptor with 5v 2A
6. USB mouse
7. USB keyboard
8. Female – Female jumper wires. (5 numbers)
9. ADS1115

Software Required:-
1. Raspbian OS
2. Matplotlib library

Procedure:-
1. Hardware Setup:-

● Connect the pins as given below....
Pi 3V to ADS1015 VDD - Remember the maximum input voltage to any
ADC channel cannot exceed this VDD 3V value!
Pi GND to ADS1015 GND
Pi SCL to ADS1015 SCL
Pi SDA to ADS1015 SDA

ADS1115 Function Pin No Function/GPIO
GND Ground 6 GND
VDD Power Supply 1 -
SDA Data bits 3 GPIO 2/ SDA
SCL Serial clock 5 GPIO 3 / SCL

2. Software Setup:-

Step 1:- Enable Raspberry Pi I2C interface
pi@raspberrypi : ~ $ sudo raspi-config
Go to Interfacing Options I2C Enable (Yes)

Step 2:- Update the Raspberry pi
pi@raspberrypi : ~ $ sudo apt-get update
pi@raspberrypi : ~ $ sudo apt-get upgrade

Step 3:- Install the Adafruit ADS1115 library for ADC
To install the dependencies starting with the Adafruit python module for the ADS115 chip,
Ensure you are in the Raspberry Pi home directory ($ cd ~)

pi@raspberrypi : ~ $ sudo apt-get install build-essential python3-dev python3-smbus git
pi@raspberrypi : ~ $ git clone https://github.com/adafruit/Adafruit_Python_ADS1x15.git

Change into the cloned file's directory and run the setup file

pi@raspberrypi : ~ $ cd Adafruit_Python_ADS1x15
pi@raspberrypi : ~/Adafruit_Python_ADS1x15 $ sudo python3 setup.py install

Step 4:- Test the library and 12C communication.
Now,it is important to test the library and ensure the ADC can communicate with the raspberry pi over I2C. To do this use an example script that comes with the library.

pi@raspberrypi : ~/Adafruit_Python_ADS1x15 $ cd examples
pi@raspberrypi : ~/Adafruit_Python_ADS1x15 /examples $ python3 simpletest.py

If the I2C module is enabled and connections are good, it should display the data as below
If an error occurs, check to ensure the ADC is well connected to the PI and I2C communication is enabled on the Pi.

Step 5:- Install Matplotlib
pi@raspberrypi : ~ $ sudo apt-get install python3-matplotlib

Step 6:- Python Code for Raspberry Pi Oscilloscope:

```
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
import Adafruit_ADS1x15
# Create an ADS1115 ADC (16-bit) instance.
adc = Adafruit_ADS1x15.ADS1115()
GAIN = 1
val = [ ]
# Start continuous ADC conversions on channel 0 using the previous gain value.
adc.start_adc(0, gain=GAIN)
print('Reading ADS1x15 channel 0')
fig, ax = plt.subplots()
ax.set_ylim(-5000,5000)
ax.set_title('Oscilloscope')
ax.grid(True)
ax.set_ylabel('ADC outputs')
line, = ax.plot([], 'ro-', label='Channel 0')
ax.legend(loc='lower right')
def update(cnt):
# Read the last ADC conversion value and print it out.
value = adc.get_last_result()
print('Channel 0: {0}'.format(value))
# Set new data to line
line.set_data(list(range(len(val))), val)
ax.relim()
```

```
ax.autoscale_view()
#Store values for later
val.append(int(value))
if(cnt>50):
val.pop(0)
ani = FuncAnimation(fig, update, interval=500)
plt.show()
```

Practical No.6

Aim: Interfacing 16X2 LCD with Raspberry Pi to display different messages

Hardware Required:-
1. Raspberry Pi 3B+
2. Ethernet Cable
3. Monitor
4. HDMI to VGI convertor (required only in case of VGA cable)
5. adapter with 5v 2A
6. USB mouse
7. USB keyboard
8. 16X2 LCD display
9. I2C module for LCD display

Procedure:-
1. Hardware Setup:

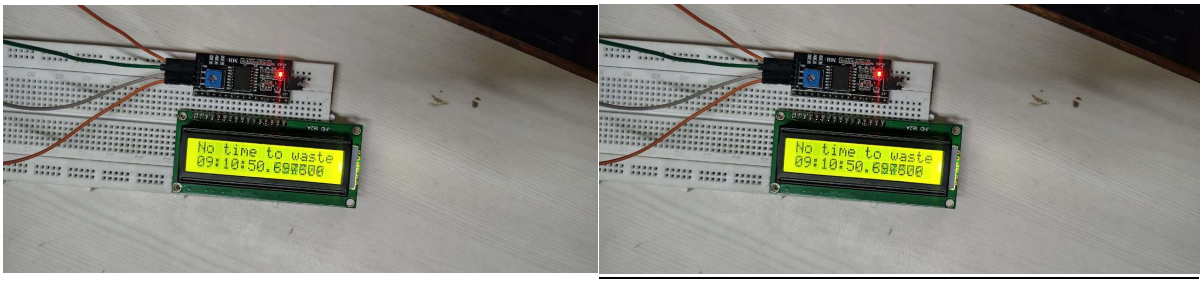
I2C LCD Function Pin No(Rasp. Pi) Function/GPIO
GND Ground 6 GND
VCC Power Supply 4 +5V
SDA Data bits 3 GPIO 2/ SDA
SCL Serial clock 5 GPIO 3 / SCL

Step 1:- Install "LCD" library
$ git clone https://github.com/sterlingbeason/LCD-1602-I2C

Step 2:- configure and verify I2C address
$ sudo raspi-config
Select : >Advaced options> Interface options> I2C>enable it nable\\
$ i2cdetect -y 1

Optional:-
$sudo apt-get install -y python3-smbus
$ sudo apt-get install -y i2c tools

Output:-



Practical No.7

Aim:- Interface RFID module with Raspberry Pi to read and write to RFID tags/cards.

Hardware Required:-
1. Raspberry Pi 3B+
2. Ethernet Cable
3. Monitor
4. HDMI to VGA convertor
5. Micro SD card (any class best is class 10)
6. Adaptor with 5v 2A
7. USB mouse
8. USB keyboard
9. Relay board
10. Female – Female jumper wires. (7 numbers)
11. Female – Male jumper wires(3 numbers)
12. RFID-RC522
13. Buzzer(optinal)
14. LED (red and green)
15. Bread board

Software Required:-
1. Raspbian OS
2. Thonny Python IDE

Procedure:-
1. Hardware Setup:

Circuit Connections
MFRC522 Raspberry Pi 3 Model B
3.3V Pin 1 (3.3V)
GND Pin 6 (GND)
SDA (SS) GPIO8 (Pin 24)
SCK GPIO11 (Pin 23)
MOSI GPIO10 (Pin 19)
MISO GPIO9 (Pin 21)
RST GPIO25 (Pin 22)

Step 1:- Update Raspberry Pi

Step 2:- Enable SPI Interface using sudo raspi-config tool
Select using Arrow Keys -> Interfacing Options-> SPI Enable

Step 3:- Reboot Raspberry Pi

Step 4:- Check to make sure that SPI has been enabled.

Step 5:- Install python3-dev, python3-pip packages to setting up RFID reader
Step 6:- Install spidev to Raspberry Pi using pip. The spidev library helps to handle interactions with the SPI
$ sudo apt-get install spidev
Or
$ pip install - -break-system-packages spidev

Step 7:- Install the MFRC522 library using pip that helps talk to the RC522 module over the SPI interface

$ git clone https://github.com/pimylifeup/MFRC522-python

Now install setup.py file available in the directory MFRC522-python by using below command ->
$ cd MFRC522-python
MFRC522-python $ sudo python setup.py install

There are two files included in this repository: MFRC522.py which is an implementation of the RFID RC522 circuit. SimpleMFRC522.py that takes the MFRC522.py file and greatly simplifies it.

Write.py:-

```
from mfrc522 import SimpleMFRC522
import RPi.GPIO as GPIO

reader = SimpleMFRC522()

try:
    text = input("Enter text to write: ")
    print("Place RFID tag near the reader...")
    reader.write(text)
    print("Data written successfully!")
finally:
    GPIO.cleanup()
```

read.py:-

```
from mfrc522 import SimpleMFRC522
import RPi.GPIO as GPIO

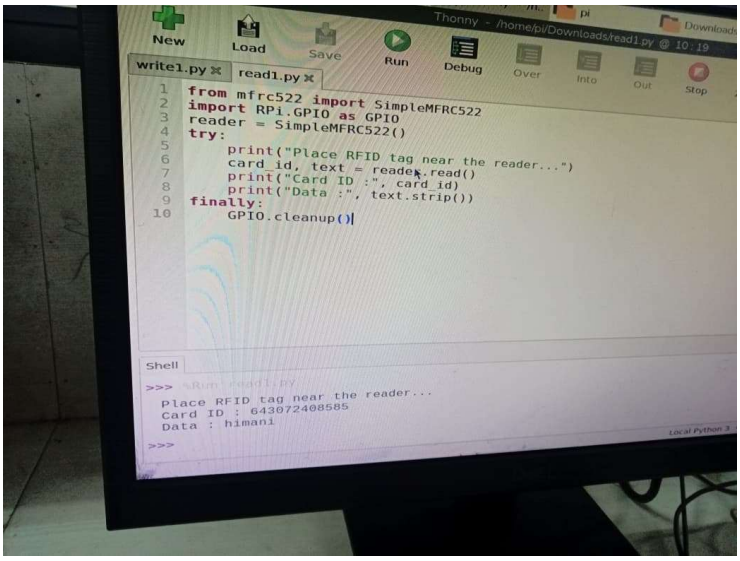
reader = SimpleMFRC522()

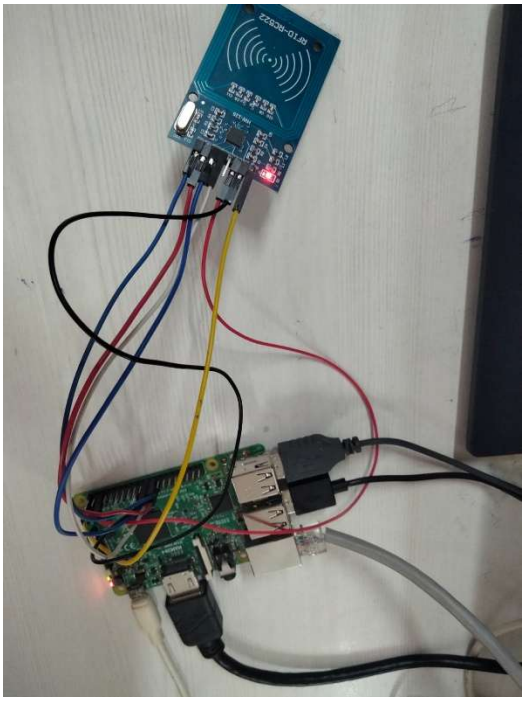
try:
    print("Place RFID tag near the reader...")
    card_id, text = reader.read()
```

```
    print("Card ID :", card_id)
    print("Data :", text.strip())

finally:
    GPIO.cleanup()
```

Output:-





Practical No.8

Aim:- Interface Fingerprint Sensor with Raspberry Pi.

1. Wiring:-

AS608	**Raspberry Pi 3B+**
VCC	5V, Pin 2
GND	GND, Pin 6
TX	GPIO15 / RXD, Pin 10
RX	GPIO14 / TXD, Pin 8

2. Enable UART:-

sudo raspi-config

Select:-

Interface Options
→ Serial Port
→ Login shell over serial: No
→ Serial port hardware: Yes

Then:-

sudo reboot

Install serial support:

sudo apt update
sudo apt install python3-serial

Check:-

ls -l /dev/serial0

If you get a permission error:

sudo usermod -aG dialout $USER
sudo reboot

3. Complete fingerprint.py

Save this as:-

fingerprint.py

```
import serial
import time
import struct


# ============================================================
# AS608 / R305 FINGERPRINT SENSOR
# Raspberry Pi 3B+ UART
# ============================================================

PORT = "/dev/serial0"
BAUDRATE = 57600
ADDRESS = 0xFFFFFFFF

# AS608 commands
CMD_GENIMG   = 0x01
CMD_IMG2TZ   = 0x02
CMD_MATCH    = 0x03
CMD_SEARCH   = 0x04
CMD_REGMODEL = 0x05
CMD_STORE    = 0x06


# ------------------------------------------------------------
# Open UART
# ------------------------------------------------------------

ser = serial.Serial(
    port=PORT,
    baudrate=BAUDRATE,
    bytesize=serial.EIGHTBITS,
    parity=serial.PARITY_NONE,
    stopbits=serial.STOPBITS_ONE,
    timeout=1
)


# ------------------------------------------------------------
# Calculate checksum
# ------------------------------------------------------------

def checksum(packet_type, length, data):

    total = packet_type
```

```
    total += (length >> 8) & 0xFF
    total += length & 0xFF

    for b in data:
        total += b

    return total & 0xFFFF


# ------------------------------------------------------------
# Send command packet
# ------------------------------------------------------------

def send_command(command_data):

    packet_type = 0x01

    length = len(command_data) + 2

    checksum_value = checksum(
        packet_type,
        length,
        command_data
    )

    packet = bytearray()

    # Header
    packet += b'\xEF\x01'

    # Address
    packet += struct.pack(">I", ADDRESS)

    # Packet type
    packet += bytes([packet_type])

    # Length
    packet += struct.pack(">H", length)

    # Command + parameters
    packet += bytes(command_data)

    # Checksum
    packet += struct.pack(">H", checksum_value)

    ser.reset_input_buffer()
    ser.write(packet)
    ser.flush()
```

```
    return read_response()


# ---------------------------------------------------------
# Read response packet
# ---------------------------------------------------------

def read_response():

    # Header
    header = ser.read(2)

    if len(header) != 2:
        return None

    if header != b'\xEF\x01':
        return None

    # Address
    address = ser.read(4)

    # Packet type
    packet_type_data = ser.read(1)

    if len(packet_type_data) != 1:
        return None

    packet_type = packet_type_data[0]

    # Length
    length_data = ser.read(2)

    if len(length_data) != 2:
        return None

    length = struct.unpack(">H", length_data)[0]

    # Data + checksum
    body = ser.read(length)

    if len(body) != length:
        return None

    data = body[:-2]
    received_checksum = struct.unpack(">H", body[-2:])[0]

    calculated_checksum = checksum(
        packet_type,
```

```
        length,
        data
    )

    if received_checksum != calculated_checksum:
        print("Checksum error")
        return None

    return data


# ---------------------------------------------------------
# Get image from finger
# ---------------------------------------------------------

def get_image():

    response = send_command([CMD_GENIMG])

    if response is None:
        return False, "No response"

    status = response[0]

    if status == 0x00:
        return True, "Image captured"

    elif status == 0x01:
        return False, "Image error"

    elif status == 0x02:
        return False, "No finger detected"

    elif status == 0x03:
        return False, "Failed to capture image"

    else:
        return False, "Error code: 0x{:02X}".format(status)


# ---------------------------------------------------------
# Convert image to template
# buffer_number:
# 1 = character buffer 1
# 2 = character buffer 2
# ---------------------------------------------------------

def image_to_template(buffer_number):
```

```
    response = send_command([
        CMD_IMG2TZ,
        buffer_number
    ])
    if response is None:
        return False

    return response[0] == 0x00


# ----------------------------------------------------------
# Combine two fingerprint templates
# ----------------------------------------------------------

def create_model():

    response = send_command([
        CMD_REGMODEL
    ])

    if response is None:
        return False

    return response[0] == 0x00


# ----------------------------------------------------------
# Store fingerprint template
# ----------------------------------------------------------

def store_model(user_id):

    response = send_command([
        CMD_STORE,
        0x01,                   # Buffer ID
        (user_id >> 8) & 0xFF,
        user_id & 0xFF
    ])

    if response is None:
        return False

    return response[0] == 0x00


# ----------------------------------------------------------
# Search fingerprint database
# ----------------------------------------------------------
```

```
def search_fingerprint():

    # Search buffer 1
    # Start ID = 0
    # Number of templates = 0x00A3 (163)
    response = send_command([
        CMD_SEARCH,
        0x01,
        0x00,
        0x00,
        0x00,
        0xA3
    ])

    if response is None:
        return None, None

    status = response[0]

    if status != 0x00:
        return None, None

    if len(response) < 5:
        return None, None

    # Matching ID
    finger_id = (response[1] << 8) | response[2]

    # Matching confidence
    confidence = (response[3] << 8) | response[4]

    return finger_id, confidence


# ============================================================
# ENROLLMENT
# ============================================================

def enroll(user_id):

    print()
    print("=" * 50)
    print("FINGERPRINT ENROLLMENT")
    print("=" * 50)

    print("Registering User ID:", user_id)

    # ------------------------------------------------
    # First scan
```

```
# ------------------------------------------------------

print()
print("Place your finger on the sensor...")

while True:
    success, message = get_image()

    if success:
        print("Fingerprint image captured.")
        break

    time.sleep(0.2)

# Convert first image
if not image_to_template(1):

    print("Failed to convert first fingerprint.")
    return False

print("First fingerprint image stored.")

# Remove finger
print()
print("Remove your finger.")

time.sleep(2)

# ------------------------------------------------------
# Wait until finger is removed
# ------------------------------------------------------

while True:

    success, message = get_image()

    if not success:
        break

    time.sleep(0.2)

# ------------------------------------------------------
# Second scan
# ------------------------------------------------------

print()
print("Place the SAME finger again...")

while True:
```

```
    success, message = get_image()

    if success:
        print("Second fingerprint image captured.")
        break

    time.sleep(0.2)

# Convert second image
if not image_to_template(2):

    print("Failed to convert second fingerprint.")
    return False

print("Second fingerprint image stored.")

# ---------------------------------------------------
# Create model
# ---------------------------------------------------

if not create_model():

    print()
    print("ERROR: Fingerprints do not match.")
    print("Please try enrollment again.")
    return False

print()
print("Fingerprint templates matched.")

# ---------------------------------------------------
# Store model
# ---------------------------------------------------

if not store_model(user_id):

    print("ERROR: Could not store fingerprint.")
    return False

print()
print("========================================")
print("ENROLLMENT SUCCESSFUL")
print("User ID:", user_id)
print("========================================")

return True
```

```
# ============================================================
# LOGIN / AUTHENTICATION
# ============================================================

def login():

    print()
    print("=" * 50)
    print("FINGERPRINT LOGIN")
    print("=" * 50)

    print()
    print("Place your finger...")

    # Wait for finger
    while True:

        success, message = get_image()

        if success:
            print("Fingerprint detected.")
            break

        time.sleep(0.2)

    # Convert image
    if not image_to_template(1):

        print("Could not process fingerprint.")
        return False

    print("Searching fingerprint database...")

    # Search
    finger_id, confidence = search_fingerprint()

    if finger_id is None:

        print()
        print("========================================")
        print("ACCESS DENIED")
        print("Fingerprint not matched.")
        print("========================================")

        return False

    print()
    print("========================================")
    print("LOGIN SUCCESSFUL")
```

```
    print("User ID:", finger_id)
    print("Confidence:", confidence)
    print("============================================")

    return True


# ============================================================
# MAIN MENU
# ============================================================

def main():

    print()
    print("############################################")
    print("# Raspberry Pi 3B+ Fingerprint System #")
    print("############################################")

    while True:

        print()
        print("1. Enroll Fingerprint")
        print("2. Fingerprint Login")
        print("3. Exit")

        choice = input("Enter choice: ").strip()

        # ----------------------------------------------------
        # Enrollment
        # ----------------------------------------------------

        if choice == "1":

            try:

                user_id = int(
                    input("Enter User ID (0-162): ")
                )

                if user_id < 0 or user_id > 162:

                    print("User ID must be between 0 and 162.")
                    continue

                enroll(user_id)

            except ValueError:

                print("Invalid User ID.")
```

```
        # ------------------------------------------------
        # Login
        # ------------------------------------------------

        elif choice == "2":

            login()

        # ------------------------------------------------
        # Exit
        # ------------------------------------------------

        elif choice == "3":

            print("Exiting...")
            break

        else:

            print("Invalid choice.")


# ============================================================
# PROGRAM START
# ============================================================

try:

    main()

except KeyboardInterrupt:

    print()
    print("Program stopped.")

finally:

    if ser.is_open:
        ser.close()
```

4. Run it

python3 fingerprint.py

You should get:

```
########################################
# Raspberry Pi 3B+ Fingerprint System #
########################################

1. Enroll Fingerprint
2. Fingerprint Login
3. Exit
Enter choice:
```

5. First enroll a user

Choose:

1

For example:

Enter User ID (0-162): 1

Place your finger on the sensor...
Fingerprint image captured.

Remove your finger.

Place the SAME finger again...
Second fingerprint image captured.

Fingerprint templates matched.

```
==========================================
ENROLLMENT SUCCESSFUL
User ID: 1
==========================================
```

6. Test login

Run the program again and select:

2

Place the enrolled finger:-

FINGERPRINT LOGIN

Place your finger...
Fingerprint detected.
Searching fingerprint database...

==========
LOGIN SUCCESSFUL
User ID: 1
Confidence: 185
==========

Practical No.9

Aim: Visitor Monitoring with Raspberry Pi and Pi Camera.

Hardware Required:-
1. Raspberry Pi 3B+
2. Ethernet Cable
3. Monitor
4. HDMI to VGI convertor
5. Micro SD card (any class best is class 10)
6. Adaptor with 5v 2A
7. USB mouse
8. USB keyboard
9. Pi Camera

Procedure:-
1. Hardware Setup:
● Connect the Pi Camera to the camera port in the Raspberry pi.

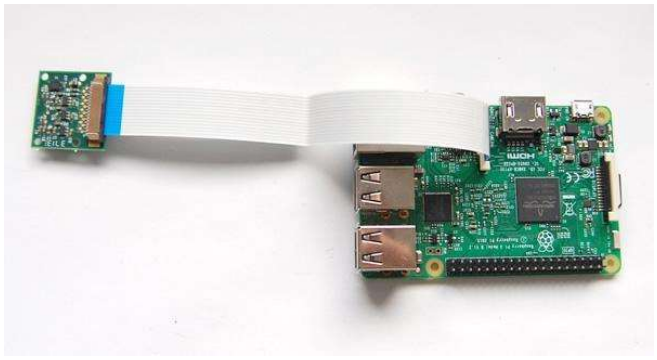
2. Software Setup:
● Start□□ Preferences □□ Raspberry Pi Configuration □□ Interfaces □□Enable Camera.

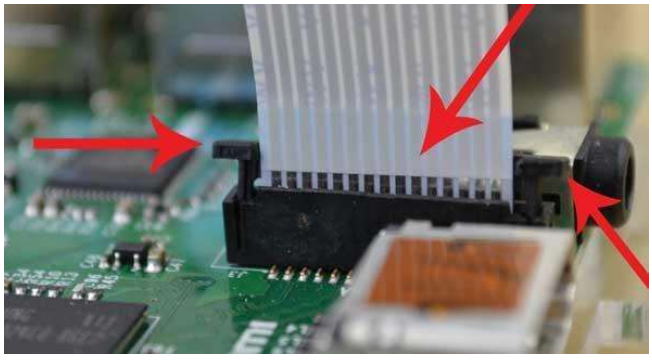
(optional)

● Reboot Raspberry pi
● Install library picamzero
sudo pip3 install picamzero
or
sudo apt install python3-picamzero

Program:-

```
from picamzero import Camera
from time import sleep
camera =Camera()
try:
camera.start_preview() #start camera
sleep(5)
camera.take_photo(&amp;#39;/home/pi/Desktop/image.jpg&amp;#39;) # take a picture
finally:
camera.stop_preview() # stop camera
```





Practical No. 10

Aim: IoT based web-based home automation using Raspberry Pi.

Hardware Required:-
1. Raspberry Pi 3B+
2. Ethernet Cable (optional)
3. Monitor
4. HDMI to VGA convertor
5. Micro SD card (any class best is class 10)
6. Adaptor with 5v 2A
7. USB mouse
8. USB keyboard
9. Relay
10. Breadboard
11. Wires (Male-Female)

Software Required:-
1. Raspbian OS

Procedure:-

Relay	Function	Pin No
GND	Ground	6
VCC	Power Supply	2
IN2	Data pin	40

2. Software Setup:
Python Code

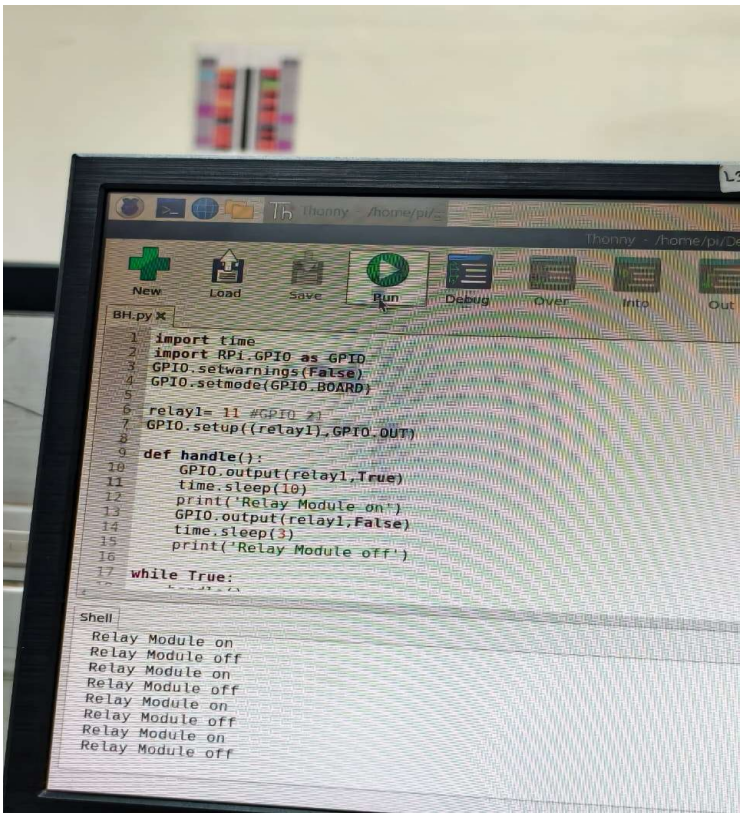
```
import time
import RPi.GPIO as GPIO
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BOARD)

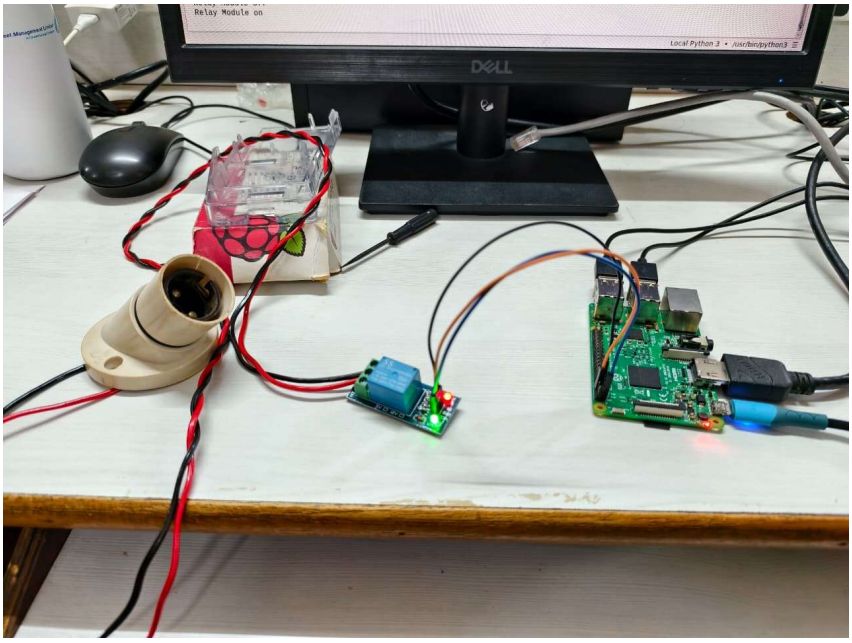
relay1= 40 #GPIO 21
GPIO.setup((relay1),GPIO.OUT)

def handle():
GPIO.output(relay1,True)
time.sleep(3)
print("Relay Module on")
GPIO.output(relay1,False)
time.sleep(3)
print("Relay Module off")

while True:
handle()
```

Output:-


New
Load
Save
Run
Debug
Over
Into
Out
BH.py
1 import time
2 import RPi.GPIO as GPIO
3 GPIO.setwarnings(False)
4 GPIO.setmode(GPIO.BOARD)
5
6 relay1= 11 #GPIO 21
7 GPIO.setup((relay1),GPIO.OUT)
8
9 def handle():
10 GPIO.output(relay1,True)
11 time.sleep(10)
12 print('Relay Module on')
13 GPIO.output(relay1,False)
14 time.sleep(3)
15 print('Relay Module off')
16
17 while True:
Shell
Relay Module on
Relay Module off
Relay Module on
Relay Module off
Relay Module on
Relay Module off
Relay Module on
Relay Module off


Relay Module on
DELL

Practical No. 1

Aim:- Write a program to check the current CPU temperature and condition using raspberry pi.

Code:-

```
import psutil
import time

print("====================================")
print("     Raspberry Pi System Monitor")
print("====================================")

try:
    while True:
        # Read CPU usage
        cpu_usage = psutil.cpu_percent(interval=1)

        # Read memory usage
        memory = psutil.virtual_memory()
        memory_usage = memory.percent

        # Read CPU temperature
        temperatures = psutil.sensors_temperatures()
        cpu_temp = None

        if "cpu_thermal" in temperatures and temperatures["cpu_thermal"]:
            cpu_temp = temperatures["cpu_thermal"][0].current
        elif "thermal_zone0" in temperatures and temperatures["thermal_zone0"]:
            cpu_temp = temperatures["thermal_zone0"][0].current
```

```
# Display system information
print("-----------------------------------")
print("CPU Usage       :", cpu_usage, "%")
print("Memory Usage    :", memory_usage, "%")

if cpu_temp is not None:
    print("CPU Temperature :", cpu_temp, "°C")
else:
    print("CPU Temperature : Not Available")

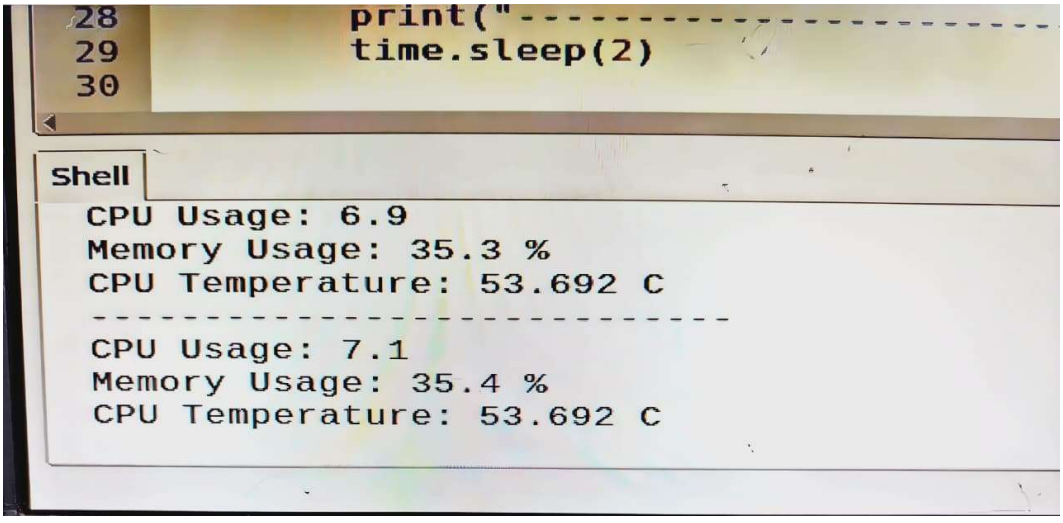
print("-----------------------------------")

time.sleep(2)
```

Open cmd and type:

- sudo apt update
- sudo apt install python3 python3-pip -y
- python3 -c "import psutil; print('CPU Usage:', psutil.cpu_percent(interval=1), '%')"
- python3 -c "import psutil; print('CPU Usage:', psutil.cpu_percent(interval=1), '%')"

Expected Output:-



Practical No.2

Aim:- Install Windows IoT 10 core on Raspberry pi and run the demo program.

Requirements

- Raspberry Pi 3B+
- microSD card, preferably **8 GB or larger**
- Windows PC
- SD-card reader
- Raspberry Pi power supply
- HDMI monitor and keyboard/network connection

Step 1: Download Windows 10 IoT Core

On the Windows PC, obtain the appropriate **Windows 10 IoT Core image/package for Raspberry Pi 3** from Microsoft's Windows IoT documentation/archive.

The image is normally provided as an .ffu file or through an installer that prepares the SD card.

Step 2: Prepare the microSD card

Insert the microSD card into the Windows PC.

Use Microsoft's supported IoT Core installation tools to write the image to the card.

If using a command-line FFU workflow, the basic operation is to apply the .ffu image to the SD card using Microsoft's deployment tools.

Warning: Writing the image will erase the contents of the microSD card.

Step 3: Insert the card into Raspberry Pi 3B+

1. Turn off the Raspberry Pi.
2. Insert the prepared microSD card.
3. Connect HDMI, keyboard and network.
4. Connect the power supply.
5. Wait for Windows 10 IoT Core to boot.

The first boot can take several minutes.

Step 4: Find the Raspberry Pi's IP address

Connect the Pi to your network using Ethernet or Wi-Fi if supported by your image.

From the Windows PC, you can check the network or use:

ping raspberrypi

If that does not work, find the Pi's IP address from your router.

Step 5: Open Windows Device Portal

On the Windows PC, open:
http://<Raspberry-Pi-IP>:8080

For example:
http://192.168.1.25:8080

Windows IoT Core's **Device Portal** allows you to manage the Raspberry Pi and deploy applications.

Step 6: Run the demo application

Windows IoT Core provides sample applications such as:

- **Hello Blinky**
- GPIO/LED samples
- Basic Universal Windows Platform (UWP) applications

For a Raspberry Pi GPIO demonstration, **Hello Blinky** is a suitable starting example. The basic flow is:

Windows PC
  |
  | Deploy UWP application
  ↓
Raspberry Pi 3B+
  |
  ↓
Windows 10 IoT Core
  |
  ↓
GPIO
  |
  ↓
LED ON/OFF

Step 7: Example demo — LED blinking

Connect an LED with a suitable resistor to a GPIO pin, for example:

Raspberry Pi 3B+

GPIO 17 (Pin 11) ── 220Ω ── LED ── GND (Pin 6)

A UWP application can then control GPIO 17:

using Windows.Devices.Gpio;

```
using System.Threading.Tasks;

GpioPin ledPin;

private void InitGPIO()
{
   var gpio = GpioController.GetDefault();

   if (gpio == null)
      return;
   ledPin = gpio.OpenPin(17);
   ledPin.SetDriveMode(GpioPinDriveMode.Output);
   ledPin.Write(GpioPinValue.Low);
}

private async Task BlinkLED()
{
   while (true)
   {
      ledPin.Write(GpioPinValue.High);
      await Task.Delay(1000);

      ledPin.Write(GpioPinValue.Low);
      await Task.Delay(1000);
   }
}
```

This produces:

```
LED ON
 ↓ 1 second
LED OFF
 ↓ 1 second
LED ON
 ↓
...
```

Expected result

After deploying and running the demo:

```
Windows 10 IoT Core
      ↓
 Raspberry Pi 3B+
      ↓
   GPIO 17
      ↓
     LED
      ↓
ON ── OFF ── ON ── OFF
```